

11 — Class Reactivity

STATE MANAGEMENT OOP

Q. How do you bundle reactive data together with the methods that modify it into a single, portable object?

ANSWER. If you are building a complex game character, having their health `$state` and their `takeDamage()` logic scattered across multiple files is an organizational nightmare. You declare class fields using the `$state` rune inside a standard JavaScript class, letting you bundle live data and complex logic into clean, reusable objects.

```
export class User {  
  name = $state('');  
  
  constructor(name) {  
    this.name = name;  
  }  
}
```

User.js

```
<script>  
  import { User } from './User.js';  
  const user = new User('Alice');  
</script>  
  
<input bind:value={user.name} />
```

Profile.svelte

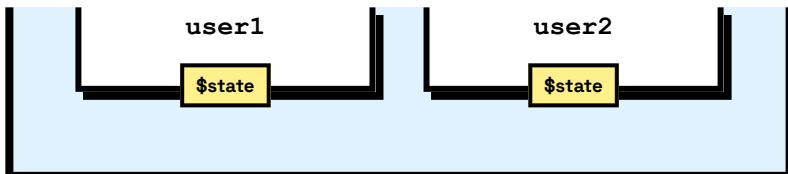
class User

INSTANCE



INSTANCE





- ① **Summary.** `$state` now works perfectly inside a JavaScript `class`. This is powerful because it lets you bundle your reactive data together with the custom logic that updates it. You can now create clean, reusable objects to pass around your app.